

1. Executive Summary

Prompt injection is a security problem in which text, images, OCR output, documents, or tool responses contain instructions that attempt to redirect an AI system away from its trusted task. This project turns that problem into a repeatable evaluation workflow.

The framework contains versioned attack and benign corpora, deterministic undefended and defended agents, a transparent detector, multimodal image testing, local and hosted model adapters, CPU-only profiles, and structured reporting.

What the project proves

- ¹ Untrusted content can be tested as a security boundary rather than treated as ordinary input.
- ¹ Defense quality can be compared using explicit metrics such as detection, bypass, accuracy, false positives, errors, confidence, and latency.
- ¹ Low-cost CPU experiments can produce useful and reproducible evidence without GPU infrastructure.
- ¹ Security evaluation is stronger when bypasses and limitations are preserved instead of hidden.

Why it is valuable as a portfolio project

2. How the System Works



Execution flow

- 1 **Bootstrap:** checks Python and Node.js, installs missing declared packages, and prepares the selected command.
- 2 **Load data:** reads a versioned text corpus, image corpus, local folder, JSON manifest, or HTTP(S) manifest.
- 3 **Establish trust boundaries:** keeps the trusted review task separate from OCR, image text, documents, and other untrusted evidence.
- 4 **Run baselines:** compares an undefended agent with a defended pipeline.
- 5 **Detect attacks:** scores overrides, forged authority, role changes, tool calls, urgency, exfiltration, and related patterns.
- 6 **Evaluate models:** sends consistent inputs to Ollama, OpenAI, Anthropic, Gemini, Hugging Face, or compatible local APIs.
- 7 **Report results:** writes structured records and generates dashboards, catalogs, and office documents.

Technologies used

PythonNode.jsOllamaLLMsVision-language modelsAI agentsPrompt engineeringThreat modelingJSON/CSVHTML reportingCPU evaluation

3. Research and Evaluation Design

Attack surface

Attack family	What it tests
Direct override	Whether the model obeys an instruction to ignore the trusted task.
Forged authority	Whether labels such as system, admin, or supervisor improperly change priority.
Delimiter escape	Whether fake document boundaries can impersonate trusted context.
Role-play and urgency	Whether fictional framing or emergency pressure bypasses verification.
Fake tool response	Whether untrusted text can imitate a trusted tool result or next action.
Exfiltration	Whether the system follows requests to export logs, secrets, or process data.
Polite low-signal attack	Whether subtle language avoids detection because it does not look aggressive.

Measured outcomes

- 1 Attack detection and attack bypass rate.
- 1 Overall accuracy and benign handling.
- 1 False positives and human-review holds.
- 1 Model confidence, provider errors, image errors, and request latency.
- 1 Per-sample prompts, model responses, run IDs, and configuration metadata.

CPU-first experiment

The CPU workflow is deliberately small and sequential. The `8gb` profile uses Moondream, one sample, and a 300-second timeout; the `16gb` profile uses Qwen 2.5 VL with two samples; the larger profile uses LLaVA with four samples and a longer timeout.

This design is valuable for scholarship and research evaluation because it makes the work accessible, controlled, and replicable without presenting a small test as a universal benchmark.

4. Evidence and Reproducibility

What a reviewer can inspect

- 1 Versioned text and image injection corpora.
- 1 Transparent detector rules and decision thresholds.
- 1 Undefended versus defended agent behavior.
- 1 Model catalogs and provider adapters.
- 1 CPU-only commands and low-memory test profiles.
- 1 Machine-readable JSON/CSV results and offline HTML dashboards.
- 1 Generated Markdown, DOCX, PDF, and PowerPoint reports.
- 1 Historical results preserved by run ID.

Reproduction commands

```
# Windows
.\run.cmd --check
.\run.cmd python image_injection_test\src\run_tests.py --cpu-only --cpu-profile 8gb --defense-mode both --defense-mode both

# macOS / Linux
./run.sh --check
./run.sh python image_injection_test/src/run_tests.py --cpu-only --cpu-profile 8gb --defense-mode both --defense-mode both
```

Important evidence links

- 1 [GitHub repository](#)
- 1 [Main technical README](#)
- 1 [CPU image testing guide](#)
- 1 [Attack and defense guide](#)

5. Resume and CV Presentation

One-line CV description

Built a cross-platform AI security framework that evaluates text and image prompt injection across LLMs and vision models using reproducible corpora, defended agent simulations, CPU-based testing, and explainable reports.

Resume entry

AegisAI Text and Image Prompt-Injection Defense Framework AI Security Research Project | Python, LLMs, Multimodal AI, Ollama, Node.js

- 1 Developed a reproducible framework for testing whether LLMs and vision-language models distinguish trusted instructions from untrusted text, OCR, and image content.
- 1 Implemented undefended and defended AI-agent simulations with transparent `allow`, `hold`, and `block` security decisions.
- 1 Designed attack scenarios covering instruction overrides, forged authority, role-play jailbreaks, fake tool responses, urgency manipulation, and data exfiltration.
- 1 Integrated local and hosted model providers and created CPU-only evaluation profiles for low-resource environments.
- 1 Generated structured JSON/CSV records and offline HTML, Markdown, DOCX, PDF, and PowerPoint reports with latency, confidence, errors, and historical run metadata.

Skills line

AI Engineering | LLM Evaluation | AI Agents | Multimodal AI | Prompt Engineering | AI Security | Threat Modeling | Python | Node.js | Ollama | Model Benchmarking | Reproducible Research

What not to claim

Do not say the project eliminates prompt injection or proves that a model is secure. Say that it evaluates defenses, measures bypasses, documents limitations, and provides evidence for follow-up research.

6. Interview Presentation Method

Use a clear five-part structure. Keep the first explanation under two minutes, then offer technical depth if the interviewer wants it.

- 1 **Problem:** "AI systems may treat instructions inside an image or document as trusted commands."
- 2 **Approach:** "I created controlled attack corpora and compared undefended and defended agents across text and image inputs."
- 3 **Engineering:** "The system uses Python for evaluation and detection, model adapters for local and hosted LLMs, Node.js for reports, and CPU profiles for low-cost reproduction."
- 4 **Evidence:** "Every run records prompts, model configuration, prediction, confidence, latency, errors, and historical output."
- 5 **Learning:** "The key lesson is that a detector alone is not enough; trusted-task enforcement provides a second defense layer when detection misses subtle attacks."

Two-minute interview answer

"My project focuses on prompt injection, where untrusted text or image content tries to control an AI system. I built a reproducible evaluation framework with attack and benign corpora, a deterministic undefended baseline, a defended agent pipeline, and model adapters for local and hosted LLMs. For images, it supports both OCR-style testing and actual image-byte evaluation. I also added CPU profiles so the experiments can run on ordinary laptops, and the system generates structured results and offline reports. The most important design decision was separating the trusted review task from untrusted content and measuring both detection and bypasses. I present it as a research prototype, not a guarantee of security, because documenting limitations is part of responsible AI engineering."

Likely questions and strong answers

Question	Answer direction
Why is this important?	LLM applications increasingly process external content, and that content can contain instructions that conflict with the trusted task.
Why use CPU tests?	They reduce cost, improve accessibility, and make the experiment easier for another researcher to reproduce.
How do you measure success?	Detection, bypass, accuracy, false positives, errors, confidence, latency, and per-sample evidence.
What are the limitations?	Corpus size, model dependence, static attacks, provider errors, and the fact that this is not a production guarantee.
What would you improve?	Add adaptive attacks, human review queues, larger multimodal corpora, statistical repetition, and stronger model-based detectors.

7. Scholarship and Research Presentation

Scholarship motivation paragraph

My project investigates a growing challenge in artificial intelligence: the ability of AI systems to distinguish trusted instructions from untrusted content. I designed a reproducible framework that tests prompt injections delivered through text, OCR transcripts, and images. The system compares undefended and defended AI-agent behavior, evaluates multiple local and hosted models, records measurable security outcomes, and produces transparent reports. The CPU-based workflow makes the research accessible on ordinary hardware, while documented limitations and preserved bypasses support honest, repeatable scientific analysis. This work has strengthened my interest in AI security, responsible machine learning, multimodal systems, and research that connects technical innovation with practical safety.

Research contribution

The contribution is not a claim that one detector solves prompt injection. The contribution is an experiment design: a trust-boundary model, a reusable attack taxonomy, paired defense conditions, low-cost reproduction, structured measurements, and explicit limitations.

How to discuss European study goals

- 1 Connect the project to responsible AI, cybersecurity, trustworthy machine learning, and human-centered technology.
- 1 Explain that Europe's research environment is attractive because it combines technical innovation with safety, privacy, governance, and societal impact.
- 1 Describe the next research step as adaptive and multimodal evaluation rather than promising a final solution.
- 1 Emphasize collaboration: the framework is designed so other researchers can reproduce, extend, and challenge the findings.

Three-minute scholarship presentation

"I became interested in how AI systems handle information that looks like an instruction but is actually untrusted data. This led me to build AegisAI, a framework for testing prompt injections in text and images. I created a versioned corpus, simulated an undefended and defended agent, integrated multiple LLM providers, and designed CPU-based experiments that can be reproduced without expensive hardware. The system measures detection, bypass, accuracy, false positives, errors, confidence, and latency, then generates transparent reports. The project taught me that responsible AI research requires more than strong average results: it requires controlled experiments, clear trust boundaries, reproducibility, and honest reporting of failure cases. At university, I would extend this work toward adaptive attacks, multimodal security, human review, and statistically stronger evaluation."

8. Development Roadmap

Stage	Planned improvement	Research value
1. Stronger evaluation	Repeat runs across more samples, models, and CPU profiles.	Improves confidence and exposes variance.
2. Adaptive attacks	Generate attack variants that respond to detector behavior.	Tests whether defenses generalize beyond known patterns.
3. Human review	Add a review queue for `hold` decisions.	Connects automated detection with human oversight.
4. Multimodal expansion	Add PDFs, screenshots, charts, QR codes, and hybrid text-image cases.	Reflects realistic AI application inputs.
5. Statistical reporting	Add confidence intervals, repeated trials, and experiment configuration manifests.	Strengthens academic and engineering conclusions.

Responsible-use statement

This framework is for authorized research, education, and controlled security testing. Attack examples should not be connected to real external tools without isolation, authorization, logging, and human review. A bypass is a research finding, not a recommendation to deploy an unsafe agent.

Final positioning

AegisAI demonstrates the complete research loop: define a security problem, design controlled attacks, build a measurable defense boundary, compare models, preserve evidence, communicate limitations, and propose the next experiment.

Portfolio source and technical documentation: [GitHub repository](#).